PHYSICAL AI AND SCIML IN THREE GOES · HANDBOOK

Go 1: Foundations

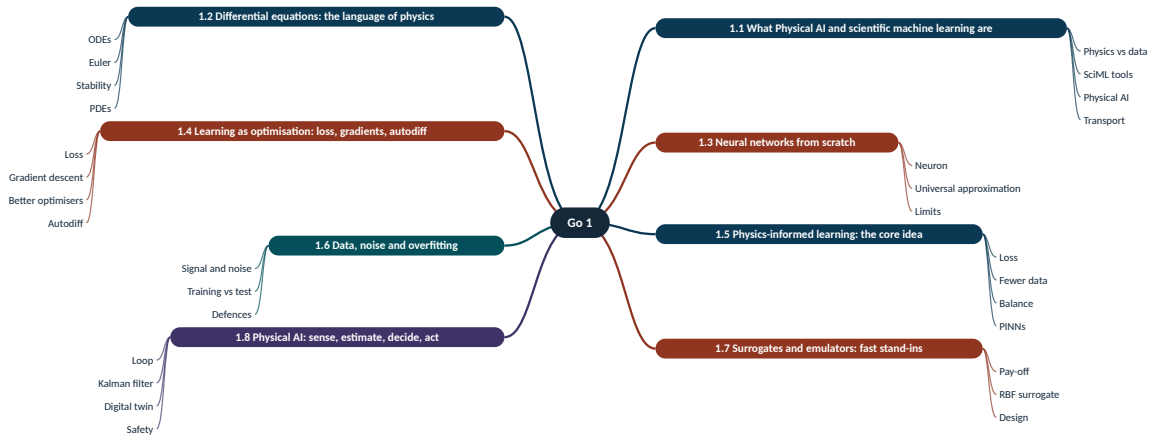
8 modules · about 5.1 hours of study

Companion to the website. Explorers, simulations, challenges and runnable code are online; this handbook keeps the text, worked numbers, mind maps and questions for reading offline.

Contents

1. **1.1 What Physical AI and scientific machine learning are** · 35 min
   Scientific machine learning blends physical laws with learning from data. Physical AI puts such models into machines that sense and act in the real world, from robots to automated vehicles.
2. **1.2 Differential equations: the language of physics** · 40 min
   A differential equation says how something changes. Solving it, with a formula or step by step with methods such as Euler's, predicts how the system evolves. Step size controls accuracy.
3. **1.3 Neural networks from scratch** · 40 min
   A neural network is a flexible function built from simple units. Each neuron applies a weighted sum and a nonlinearity; stacked together they can approximate almost any smooth relationship.
4. **1.4 Learning as optimisation: loss, gradients, autodiff** · 40 min
   Training a model means minimising a loss by walking downhill along its gradient. Learning rate, momentum and local minima decide whether training succeeds; automatic differentiation computes the gradients.
5. **1.5 Physics-informed learning: the core idea** · 40 min
   A physics-informed model is trained to fit the data and to obey a known equation at many points. The physics fills the gaps between and beyond measurements, so far fewer data are needed.
6. **1.6 Data, noise and overfitting** · 35 min
   Real data are noisy and limited. Models that are too flexible learn the noise; models that are too simple miss the pattern. Judge every model on data it has not seen.
7. **1.7 Surrogates and emulators: fast stand-ins** · 35 min
   A surrogate learns the input–output map of a slow simulator from a few runs, then answers new questions almost instantly. Where you sample and what you assume about smoothness decide its accuracy.
8. **1.8 Physical AI: sense, estimate, decide, act** · 40 min
   Physical AI closes a loop: sensors measure the world, filters estimate its state, models predict, controllers act, and the world responds. Physics-based estimation such as the Kalman filter is the foundation.

Mind map of the go



1.1 What Physical AI and scientific machine learning are

35 minutes · Big picture

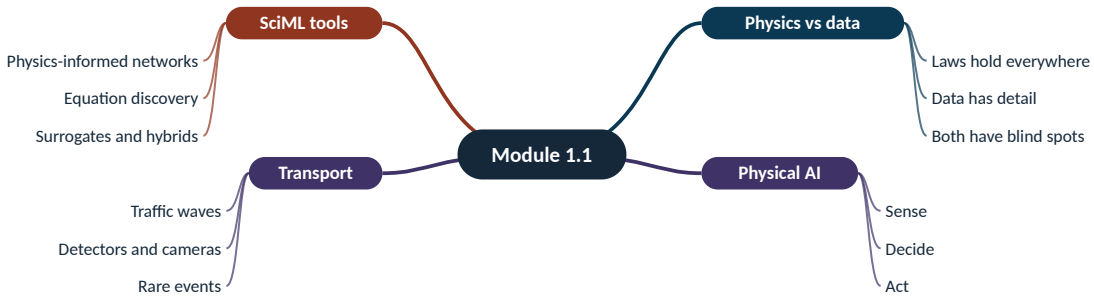
Scientific machine learning blends physical laws with learning from data. Physical AI puts such models into machines that sense and act in the real world, from robots to automated vehicles.

By the end you can

- say what scientific machine learning (SciML) and Physical AI mean, in one sentence each
- place a model on the spectrum from physics-based to data-driven
- explain why pure data fits fail outside the data and pure physics misses details
- name three transport problems where SciML helps

Overview

Engineers have always had two ways to predict the world. The first is physics: write down the laws, such as conservation of mass or Newton's laws, and solve them. The second is data: collect measurements and fit a pattern. Physics models extrapolate well but miss details we did not model. Data models capture details but fail where they have no data. Scientific machine learning, or SciML, combines the two. It builds machine learning models that respect physical laws, discovers equations from data, and builds fast surrogates of slow simulations. Physical AI goes one step further: it puts such models inside machines that sense and act in the physical world, such as robots, drones and automated vehicles, where mistakes have physical consequences. In transport, SciML estimates traffic from a few sensors, predicts crash risk, emulates climate and energy models, and runs digital twins of cities. In this module you will slide a model from pure data to pure physics and see where each one breaks.



Two ways to predict

Physics-based models start from laws: mass is conserved, force equals mass times acceleration, heat flows from hot to cold. They work even in situations never measured, but only describe what we put in. **Data-driven models** learn patterns from examples. They capture details nobody wrote down, but they can be badly wrong outside the range of their data.

Scientific machine learning

Scientific machine learning (SciML) combines the two. Its main tools, which you will meet in this course:

- **Physics-informed neural networks**: networks trained to fit data *and* obey an equation.
- **Equation discovery**: finding the law itself from data.
- **Operator learning and surrogates**: fast stand-ins for slow simulators.
- **Hybrid models**: physics plus a learned correction.
- **Uncertainty quantification**: saying how sure we are.

TRY Try it

In the explorer, set trust in physics to 0 and then to 1. Where does each model fail? Can a value in between beat both?

Physical AI

Physical AI is artificial intelligence that perceives and acts in the physical world: robots, drones, automated vehicles, smart traffic signals. Its models must respect physics (a car cannot stop instantly), run in real time and be safe when wrong. That is why SciML and Physical AI belong together: physical structure makes learned models more reliable.

Why transport needs it

Transport is full of physics (traffic waves, vehicle dynamics, energy) and full of data (detectors, cameras, phones), but data are always sparse where it matters: during crashes, at extreme densities, in future climates. Go 3 applies SciML to traffic estimation, automated vehicles, safety, energy and emissions, integrated assessment models such as GCAM, crowds and digital twins.

RESEARCH Research angle

Many transport problems are 'small data' problems in disguise: plenty of normal data, very little about the rare events we care about. Physics is how we bridge that gap.

LAB Data fit versus physics

A degree-5 polynomial fits 8 points well but explodes after t = 2; the slightly wrong physics model stays close.

```
data fit error beyond the data: 259.02
physics error beyond the data:  0.13
```

Takeaways

- Physics extrapolates but misses details; data captures details but fails outside its range.
- Scientific machine learning combines physical laws with learning from data.
- Physical AI puts models into machines that sense and act in the real world.
- Transport is physics-rich but data-sparse where it matters most.

Check yourself

1. A pure data-driven model is most likely to fail…
   (a) Inside the range of its data · (b) Far outside the range of its data · (c) Never · (d) Only with clean data
2. Scientific machine learning mainly aims to…
   (a) Replace physics with data · (b) Combine physical laws with learning from data · (c) Collect more data · (d) Make models slower
3. Which is an example of Physical AI?
   (a) A spam filter · (b) An automated vehicle · (c) A spreadsheet · (d) A search engine
4. Why do transport problems need physics?
   (a) Data are always perfect · (b) Data are sparse exactly where it matters, such as rare events · (c) Physics is cheaper than data · (d) Laws change every year

Answers: 1 (b) Data models can be badly wrong where they have no data.; 2 (b) It uses both.; 3 (b) It senses and acts in the physical world.; 4 (b) Physics fills gaps in data.

Words

- **Scientific machine learning (SciML)**: Machine learning methods that combine data with scientific knowledge such as physical laws.
- **Physical AI**: AI systems that perceive and act in the physical world, such as robots and automated vehicles.
- **Data-driven model**: A model learned from examples without built-in laws.
- **Physics-based model**: A model built from physical laws and equations.
- **Extrapolation**: Predicting outside the range of the data used to build a model.

Reading

- Baker et al. (2019): Basic Research Needs for Scientific Machine Learning (US DOE): The report that named and framed the field. https://doi.org/10.2172/1478744
- Karniadakis et al. (2021): Physics-informed machine learning: A readable review in Nature Reviews Physics. https://doi.org/10.1038/s42254-021-00314-5
- Steve Brunton: Physics-informed machine learning (video course): Clear lectures on SciML for beginners. https://www.youtube.com/@Eigensteve

1.2 Differential equations: the language of physics

40 minutes · Foundations

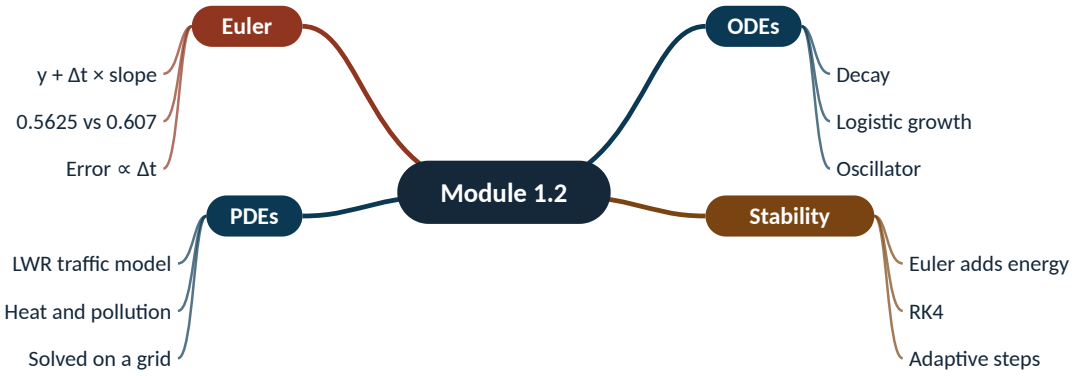
A differential equation says how something changes. Solving it, with a formula or step by step with methods such as Euler's, predicts how the system evolves. Step size controls accuracy.

By the end you can

- read a simple ordinary differential equation as a rule for change
- solve decay and growth equations step by step with Euler's method
- explain how the time step affects accuracy and stability
- recognise ODEs and PDEs in traffic, energy and climate models

Overview

Most laws of physics are written as differential equations. A differential equation does not say what a quantity is; it says how fast it changes. For example, a pollutant that decays at a rate proportional to how much is left follows dy over dt equals minus k times y. A queue that grows when arrivals beat departures follows a similar rule. To predict the future, we solve the equation. Sometimes there is a formula, such as an exponential decay. Usually there is not, so we step forward in time: take the current value, compute the rate of change, and move a small step. This is Euler's method, the simplest numerical solver. Small steps are accurate but slow; big steps are fast but wrong, and for some equations they even become unstable. Equations with one variable changing in time are ordinary differential equations, or ODEs. Equations that change in time and space, such as traffic waves or heat, are partial differential equations, or PDEs. In this module you will solve three equations step by step.



A rule for change

An ordinary differential equation (ODE) gives the rate of change of a quantity as a function of the quantity itself. Three examples you will use:

- **Decay**: dy/dt = −k y. Each second a fixed fraction disappears (pollutants, radioactivity, fading signals).
- **Logistic growth**: dy/dt = k y (1 − y). Growth that slows near a limit (adoption of electric cars).

- **Oscillator**: $d^2x/dt^2 = -\omega^2 x$. A spring, a swaying bridge, a car suspension.

Euler's method by hand

Take the decay law with k = 0.5 and y(0) = 1, step Δt = 0.5 s:

$$y_1 = y_0 + \Delta t \times (-k\, y_0) = 1 - 0.5 \times 0.5 \times 1 = 0.75$$

Then $y_2 = 0.75 - 0.25 \times 0.75 = 0.5625$. The exact answer at t = 1 s is $e^{-0.5} \approx 0.607$, so the error after two steps is about 0.045. Halve the step and the error roughly halves: Euler's method is **first-order accurate**.

TRY Try it

In the explorer, choose the oscillator and increase Δt. Watch Euler's swing grow. Then make Δt small: does the growth vanish, or just slow down?

Stability and better solvers

For the oscillator, explicit Euler adds a little energy every step, so the swing grows forever. Large steps make things worse, and for stiff problems the solution can explode. Higher-order methods such as Runge–Kutta (RK4) sample the slope four times per step and are far more accurate for the same work; adaptive solvers choose the step size automatically. Every SciML method in this course rests on solvers like these.

TRAP Common trap

A solver that 'runs' is not necessarily right. Always halve the step and check that the answer barely changes.

PDEs in transport

When a quantity varies along a road or through space, we get a partial differential equation. The **LWR traffic model** (Lighthill, Whitham and Richards) says vehicles are conserved: $\partial\rho/\partial t + \partial(\rho v)/\partial x = 0$. The heat equation, air-pollution dispersion and the shallow-water equations behind flood models are PDEs too. You will solve one in Go 2 and use LWR in Go 3.

LAB Euler's method for decay

The error roughly halves when the step halves: first-order accuracy.

```
Euler y(10) = 0.00317, exact = 0.00674
dt = 0.500: error 3.57e-03
dt = 0.250: error 1.95e-03
dt = 0.125: error 1.01e-03
```

Takeaways

- A differential equation is a rule for change.
- Euler's method follows the rule in small steps: y + Δt × slope.
- Smaller steps are more accurate; large steps can be unstable.
- PDEs describe change in space and time, such as traffic waves and heat.

Check yourself

1. dy/dt = −k y describes…
   (a) Steady growth · (b) Exponential decay · (c) An oscillation · (d) A constant
2. One Euler step computes…
   (a) y + Δt × slope · (b) y × Δt · (c) The exact solution · (d) y − slope
3. Halving the time step in Euler's method roughly…
   (a) Doubles the error · (b) Halves the error · (c) Leaves the error unchanged · (d) Removes all error
4. The LWR traffic model is a…
   (a) ODE · (b) PDE · (c) Neural network · (d) Statistical test

Answers: 1 (b) Each moment a fixed fraction disappears.; 2 (a) Follow the slope for a small step.; 3 (b) It is first-order accurate.; 4 (b) Density changes in space and time.

Words

- **Ordinary differential equation (ODE)**: An equation for how a quantity changes with one variable, usually time.
- **Partial differential equation (PDE)**: An equation for how a quantity changes in more than one variable, such as space and time.
- **Euler's method**: The simplest step-by-step solver: new value = old value + step × rate.
- **Time step**: The size of each step a solver takes.
- **Initial condition**: The value of the quantity at the start.

Reading

- 3Blue1Brown: Differential equations, a tourist's guide: A beautiful visual introduction. https://www.3blue1brown.com/lessons/differential-equations
- Strang: Differential Equations and Linear Algebra (MIT OpenCourseWare): Short videos with Cleve Moler on solvers. https://ocw.mit.edu/courses/res-18-009-learn-differential-equations-up-close-with-gilbert-strang-and-cleve-moler-fall-2015/
- Lighthill and Whitham (1955): On kinematic waves II: The origin of the LWR traffic model. https://doi.org/10.1098/rspa.1955.0089

1.3 Neural networks from scratch

40 minutes · Foundations

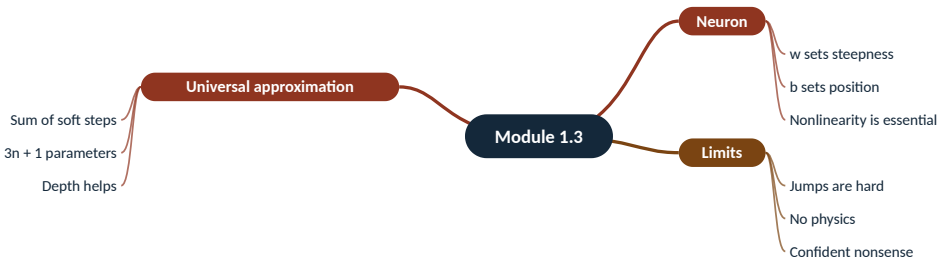
A neural network is a flexible function built from simple units. Each neuron applies a weighted sum and a nonlinearity; stacked together they can approximate almost any smooth relationship.

By the end you can

- describe a neuron as a weighted sum plus an activation function
- explain why more neurons let a network fit more complex shapes
- count the parameters of a small network
- say what a network needs to learn: data, a loss and an optimiser

Overview

A neural network is just a function with many adjustable knobs. The basic unit is a neuron. It takes inputs, multiplies each by a weight, adds them up with a bias, and passes the result through a nonlinear function such as the hyperbolic tangent, which looks like a soft step. One neuron can only make one soft step. But add many neurons side by side, each with its own position and steepness, and add their outputs with new weights, and you can build almost any smooth curve. This is the universal approximation idea. Stack layers and the network can build features of features. Learning means adjusting the weights so that the network's output matches the data, which is the topic of the next module. Networks are powerful but have no idea about physics: they only know the examples they saw. In this module you will add neurons one by one and watch a network bend into the shape of a function, and see what it struggles with.



A single neuron

A neuron computes `a = tanh(w·x + b)`. The weight w sets how steep the step is; the bias b sets where it is. The hyperbolic tangent squeezes any number into the range −1 to 1. Other common activation functions are the ReLU, max(0, z), and the sigmoid. Without a nonlinear activation, any stack of layers would collapse into a single straight line.

Many neurons, many shapes

A layer of n neurons followed by a weighted sum gives $y = \Sigma\ v_j\ \tanh(w_j\ x + b_j) + c$. With enough neurons this can approximate any continuous function on a bounded range (the **universal approximation theorem**, Cybenko 1989; Hornik 1991). Deeper networks (more layers) often need far fewer neurons for complex functions.

Parameter count for one input, n hidden neurons, one output: n weights + n biases + n output weights + 1 output bias = **3n + 1**.

TRY Try it

Fit the smooth wave with 1, 2, 4 and 8 neurons. How many do you need before the error stops improving? Then try the jump.

What networks cannot do alone

A network learns only from its examples. It struggles with sharp jumps (smooth neurons blur them), knows nothing outside the data range, and can give confident nonsense. It also has no built-in respect for physics: a network trained on traffic speeds can predict negative speeds. Adding physics, in Go 1 module 1.5 and throughout Go 2, is how SciML fixes this.

TRAP Teaching shortcut

In this explorer only the output weights are fitted; the hidden weights are random. That is a real method (random features), but full networks train every weight.

LAB A neural network forward pass with NumPy

Build the hidden layer, fit the output layer exactly, and measure the error.

```
parameters: 25
training error: 0.0037
```

Takeaways

- A neuron computes a weighted sum and applies a nonlinear activation.
- Many neurons together can approximate any smooth function.
- A one-hidden-layer network with n neurons has 3n + 1 parameters.
- Networks know only their data and nothing about physics.

Check yourself

1. A neuron computes…
   (a) A sorted list · (b) An activation of a weighted sum plus bias · (c) The derivative of its input · (d) Only additions
2. Why do networks need nonlinear activations?
   (a) To run faster · (b) Without them, layers collapse into one linear function · (c) To use less memory · (d) They do not need them
3. How many parameters has a network with 1 input, 10 hidden neurons and 1 output?
   (a) 11 · (b) 21 · (c) 31 · (d) 100

4. A network trained only on light-traffic data may…
   (a) Always be right in heavy traffic · (b) Predict physically impossible values in heavy traffic · (c) Refuse to answer · (d) Discover physics by itself

Answers: 1 (b) $a = \sigma(w \cdot x + b)$.; 2 (b) Linear of linear is linear.; 3 (c) $3 \times 10 + 1 = 31$.; 4 (b) It knows only its examples.

Words

- **Neuron**: A unit computing an activation of a weighted sum of inputs plus a bias.
- **Activation function**: The nonlinear function in a neuron, such as tanh or ReLU.
- **Weight**: An adjustable multiplier on a connection.
- **Bias**: An adjustable offset added inside a neuron.
- **Universal approximation**: The result that networks with enough neurons can approximate any continuous function on a bounded range.

Reading

- 3Blue1Brown: Neural networks: The clearest visual introduction. https://www.3blue1brown.com/topics/neural-networks
- Nielsen: Neural Networks and Deep Learning (free book): Chapter 4 shows universal approximation visually. http://neuralnetworksanddeeplearning.com/
- TensorFlow Playground: Train a small network in your browser. https://playground.tensorflow.org/

1.4 Learning as optimisation: loss, gradients, autodiff

40 minutes · Foundations

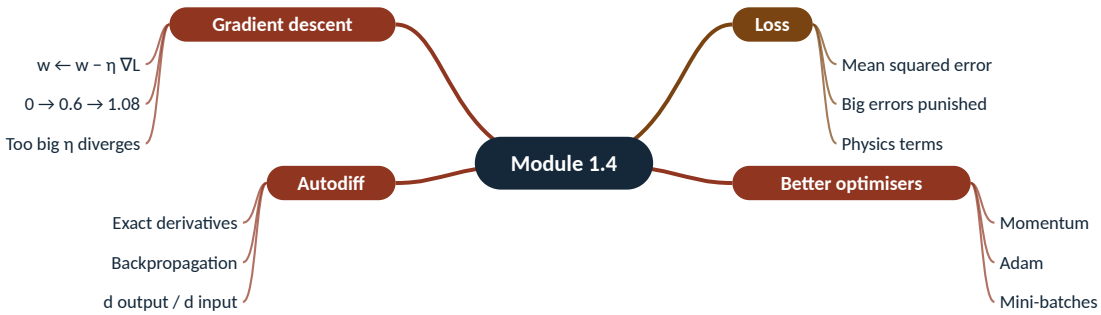
Training a model means minimising a loss by walking downhill along its gradient. Learning rate, momentum and local minima decide whether training succeeds; automatic differentiation computes the gradients.

By the end you can

- define a loss function and explain why we minimise it
- carry out gradient descent steps by hand
- explain the effect of the learning rate and momentum
- say what automatic differentiation does and why SciML depends on it

Overview

How does a network learn? We define a loss: a single number that measures how wrong the model is, such as the average squared difference between predictions and data. Learning means finding the weights that make the loss small. Imagine the loss as a landscape of hills and valleys over all possible weights. Gradient descent stands on this landscape, feels which way is downhill, and takes a step. The gradient tells the direction, and the learning rate sets the step size. Too small and training crawls. Too large and it jumps across valleys or even flies off. Real landscapes have many valleys, called local minima, and momentum, which keeps some of the previous step, can help roll past small ones. Popular optimisers such as Adam adapt the step for each weight. Computing gradients by hand is impossible for millions of weights, so software uses automatic differentiation, which applies the chain rule exactly. SciML uses the same machinery to differentiate a network with respect to its inputs, which is how physics enters the loss.



Loss functions

A loss turns 'how wrong' into one number. The most common is the **mean squared error**:

$$L(w) = (1/N) \Sigma (\hat{y}_i(w) - y_i)^2$$

Squaring makes all errors positive and punishes big ones more. In SciML we add terms that measure how badly the model breaks a physical law; minimising the total balances data and physics.

Gradient descent by hand

Take L(w) = (w − 3)², gradient 2(w − 3), start w = 0, learning rate η = 0.1:

$$w_1 = 0 - 0.1 \times 2 \times (0 - 3) = 0.6;\ w_2 = 0.6 - 0.1 \times 2 \times (0.6 - 3) = 1.08$$

Each step closes 20% of the gap to the minimum at w = 3. With η = 1.0 the step overshoots to w = 6, then back to 0: it never settles. With η above 1.0 it diverges.

TRY Try it

In the explorer, find a learning rate that reaches the deep valley near w = 4.3 without momentum. Then set momentum to 0.9 with a small learning rate.

Momentum, Adam and local minima

Real losses are bumpy, with many valleys. **Momentum** keeps a fraction of the last step, like a ball rolling, so it can pass small bumps and move faster along gentle slopes. **Adam** (Kingma and Ba 2015) adapts the step size for each weight and is the default for training physics-informed networks. Stochastic gradient descent uses random mini-batches of data, which adds helpful noise.

Automatic differentiation

Automatic differentiation (autodiff) records every arithmetic operation in the model and applies the chain rule backwards (backpropagation). It gives exact derivatives, not finite-difference approximations, at a cost similar to one extra model evaluation. Crucially for SciML, autodiff can also give the derivative of the network's output with respect to its *input*, such as dŷ/dt, which is exactly what a differential equation needs.

MODEL How SciML uses it

In a physics-informed network, autodiff gives dŷ/dt at any time t; the loss then penalises dŷ/dt + k ŷ ≠ 0. You will build exactly this in module 1.5 and Go 2.

LAB Gradient descent and a numerical gradient check

Small rates converge, 1.0 oscillates forever, above 1.0 diverges. Autodiff gives the exact gradient that finite differences only approximate.

```
learning rate 0.1: w after 20 steps = 2.965
learning rate 0.5: w after 20 steps = 3.000
learning rate 1.0: w after 20 steps = 0.000
learning rate 1.1: w after 20 steps = -112.013
gradient at w=1: exact -4.0, finite difference -4.0000
```

Takeaways

- A loss turns 'how wrong' into one number; training minimises it.

- Gradient descent steps downhill: $w \leftarrow w - \eta \times \text{gradient}$.
- The learning rate must be neither too small nor too large; momentum and Adam help.
- Automatic differentiation computes exact gradients, including with respect to inputs.

Check yourself

1. Gradient descent moves the weights…
   (a) Uphill · (b) In the direction that reduces the loss fastest · (c) Randomly · (d) To zero
2. A learning rate that is too large may…
   (a) Converge faster always · (b) Overshoot and diverge · (c) Have no effect · (d) Make the loss exactly zero
3. Momentum helps because it…
   (a) Removes the need for gradients · (b) Keeps part of the previous step, rolling past small bumps · (c) Makes the loss convex · (d) Adds more data
4. Automatic differentiation computes…
   (a) Approximate derivatives by finite differences · (b) Exact derivatives by applying the chain rule · (c) Integrals · (d) Random numbers

Answers: 1 (b) Against the gradient.; 2 (b) Steps jump across the valley.; 3 (b) Like a rolling ball.; 4 (b) Chain rule, exactly.

Words

- **Loss function**: A number measuring how wrong a model is; training minimises it.
- **Gradient**: The vector of derivatives of the loss with respect to each parameter.
- **Learning rate**: The step size of gradient descent.
- **Momentum**: Carrying over part of the previous update to smooth and speed up descent.
- **Automatic differentiation**: Computing exact derivatives of a program by applying the chain rule to its operations.

Reading

- Kingma and Ba (2015): Adam: The optimiser used in most SciML training. https://arxiv.org/abs/1412.6980
- Baydin et al. (2018): Automatic differentiation in machine learning: a survey: How autodiff works, clearly explained. https://www.jmlr.org/papers/v18/17-468.html
- Distill: Why momentum really works: Interactive explanation of momentum. https://distill.pub/2017/momentum/

1.5 Physics-informed learning: the core idea

40 minutes · Core idea

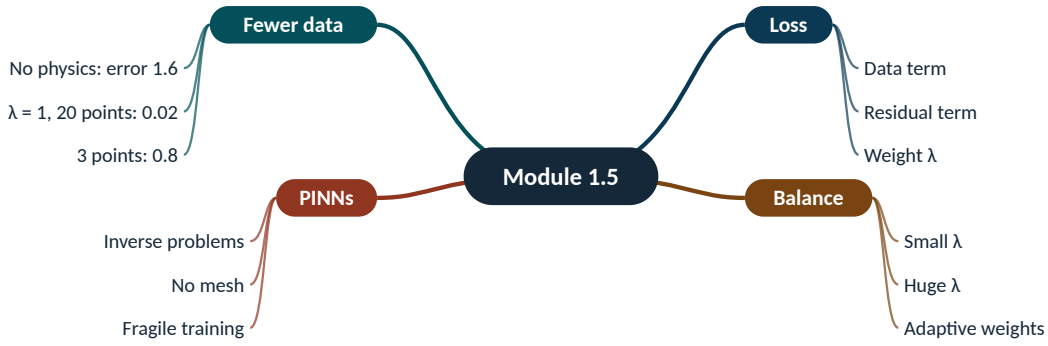
A physics-informed model is trained to fit the data and to obey a known equation at many points. The physics fills the gaps between and beyond measurements, so far fewer data are needed.

By the end you can

- write a physics-informed loss with a data term and a residual term
- explain what collocation points are and why they help
- show how the physics weight changes predictions beyond the data
- name the main strengths and weaknesses of physics-informed neural networks

Overview

This is the central idea of the course. Suppose we know the law that governs a system, say that a quantity decays as dy over dt equals minus k y, but we have only a few noisy measurements, all from the first second. A plain network fitted to those points can do anything afterwards. A physics-informed model adds a second term to the loss: at many chosen times, called collocation points, it checks how badly the model breaks the equation, and penalises that. The model must now fit the data and obey the law. Because collocation points cost nothing, we can place them everywhere, including where we have no data, and the physics fills the gap. This is the idea behind physics-informed neural networks, or PINNs, introduced by Raissi, Perdikaris and Karniadakis in 2019. The balance between the two loss terms matters: too little physics and the model wanders; too much and it ignores the data. In this module you will turn the physics weight up and down and watch the prediction lock onto the law.



Two terms in the loss

For the law dy/dt = −k y, a physics-informed loss is

$$L = (1/N_d)\ \Sigma\ (\hat{y}(t_i) - y_i)^2 + \lambda\ (1/N_c)\ \Sigma\ (d\hat{y}/dt(t_c) + k\ \hat{y}(t_c))^2$$

The first term fits N_d measurements; the second checks the law at N_c collocation points. Initial or boundary conditions can be added as extra data-like terms.

Why it needs fewer data

In the explorer, six measurements from 0 to 1 s alone leave the model free to do anything from 1 to 5 s (error about 1.6). Add 20 collocation points with $\lambda = 1$ and the error beyond the data falls to about 0.02: the law has replaced the missing measurements. With only 3 collocation points the gaps between them are unconstrained again (error about 0.8).

TRY Try it

Start at $\lambda = 10^{-4}$, then step it up. At what value does the prediction beyond the data snap onto the true curve?

Balancing the terms

If λ is tiny, the physics is ignored. If λ is huge, the optimiser satisfies the law but may drift from the data (in the explorer, $\lambda = 100$ raises the error to about 0.17, because the noisy data and the initial value are underweighted). In practice the terms are scaled, or adaptive weighting schemes are used (Wang, Teng and Perdikaris 2021).

Strengths and limits of PINNs

Strengths: work with sparse, noisy data; no mesh needed; can solve *inverse problems* (find unknown parameters such as k from data). **Limits**: training can be slow and fragile, especially for sharp fronts such as shock waves; for simple forward problems classical solvers are usually faster and more accurate. PINNs shine when data and physics must be combined.

RESEARCH Research angle

Physics-informed deep learning for traffic (Shi et al. 2021) and for crowds builds on exactly this loss. You will apply it in Go 3.

LAB A physics-informed fit with least squares

The same data, two physics weights. Because the model is linear in its weights, the loss is minimised exactly.

```
physics weight 0.0001: error beyond the data 0.154
physics weight 1: error beyond the data 0.020
```

Takeaways

- A physics-informed loss adds a penalty for breaking the governing equation.
- Collocation points are free, so physics can fill gaps where there are no data.
- The physics weight λ must balance data and law.
- PINNs suit sparse data and inverse problems; classical solvers are better for simple forward problems.

Check yourself

1. A collocation point is where…
   (a) A measurement was taken · (b) The model is checked against the governing equation · (c) Training stops · (d)

The network has a neuron

2. Why can physics-informed models work with few data?
   (a) They ignore data · (b) The equation constrains the model where there are no data · (c) They use bigger networks · (d) They need no training
3. What happens with a very large physics weight?
   (a) Perfect fit to data · (b) The law is obeyed but the data may be underweighted · (c) Training becomes instant · (d) Nothing
4. An inverse problem means…
   (a) Solving backwards in time · (b) Finding unknown parameters of the equation from data · (c) Reversing the network · (d) Using negative data

Answers: 1 (b) No measurement needed.; 2 (b) Physics fills gaps.; 3 (b) Balance matters.; 4 (b) For example finding k.

Words

- **Physics-informed neural network (PINN)**: A neural network trained with a loss that includes the residual of a governing equation.
- **Residual**: How much a candidate solution fails to satisfy an equation.
- **Collocation point**: A point where the equation residual is evaluated during training.
- **Physics weight**: The factor λ that balances the physics term against the data term.
- **Inverse problem**: Inferring unknown parameters or inputs of a model from observations.

Reading

- Raissi, Perdikaris and Karniadakis (2019): Physics-informed neural networks: The paper that launched PINNs. https://doi.org/10.1016/j.jcp.2018.10.045
- Ben Moseley: So, what is a physics-informed neural network?: A clear blog post with code. https://benmoseley.blog/my-research/so-what-is-a-physics-informed-neural-network/
- Wang, Teng and Perdikaris (2021): Gradient pathologies in PINNs: Why balancing the loss matters. https://doi.org/10.1137/20M1318043

1.6 Data, noise and overfitting

35 minutes · Foundations

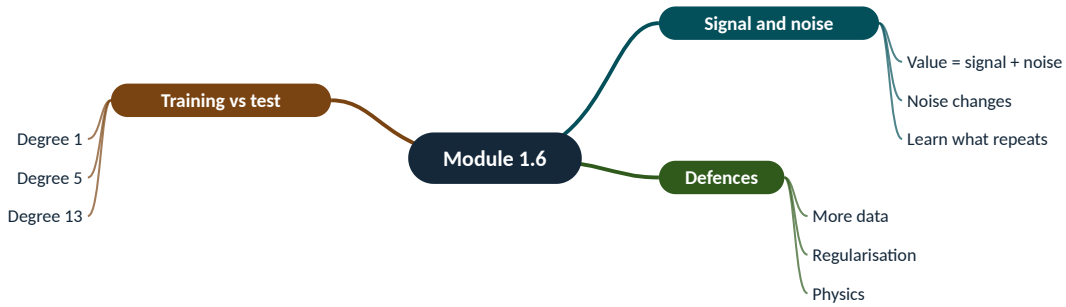
Real data are noisy and limited. Models that are too flexible learn the noise; models that are too simple miss the pattern. Judge every model on data it has not seen.

By the end you can

- explain underfitting and overfitting with an example
- separate training error from test error
- describe how noise and data quantity change the best model complexity
- list simple defences: validation, regularisation and physics

Overview

Every measurement contains noise: sensor errors, rounding, random fluctuations. A model has to learn the pattern without learning the noise. If the model is too simple, say a straight line through a wave, it misses the pattern: this is underfitting. If it is too flexible, say a polynomial of degree thirteen through fifteen points, it threads every noisy point and wiggles wildly in between: this is overfitting. The training error keeps falling as the model grows, but the error on new data first falls and then rises. The best model sits in between, and where exactly depends on how much data and how much noise we have. The only honest judge is data the model has not seen: a validation or test set. Common defences against overfitting include more data, simpler models, regularisation, which penalises large weights, and early stopping. In scientific machine learning, physics is a powerful regulariser: an equation forbids many of the wiggles that noise would otherwise produce. In this module you will push a model from underfitting to overfitting.



Signal plus noise

Think of every data point as **true value + noise**. If you measure the same traffic flow twice, you get two slightly different numbers. A model should capture what repeats (the signal), not what does not (the noise). With 15 points of a sine wave and noise of 0.15, a cubic polynomial captures the wave well; a straight line cannot.

Training error lies

In the explorer, raising the polynomial degree from 1 to 13 lowers the training error steadily (0.48 → 0.06), but the error against the true curve first falls (to about 0.09 at degree 5) and then shoots up (to about 1.1 at degree 13). A model can look perfect on its own training data and be useless. Always keep data aside for testing.

TRY Try it

Set degree 9 and increase the number of data points from 10 to 40. What happens to overfitting?

Defences

More data lets flexible models settle down. **Regularisation** adds a penalty such as $\lambda\|w\|^2$ to the loss, discouraging extreme weights. **Early stopping** halts training when validation error starts to rise. **Physics** is the SciML defence: an equation residual penalises exactly the wiggles that noise creates, because real systems do not wiggle like that.

TRAP Common trap

Tuning a model again and again on the same test set slowly leaks the test set into the model. Keep a final test set untouched until the very end.

LAB Training versus test error

Training error always falls with degree; test error has a minimum.

```
degree  1: train 0.500, test 0.467
degree  3: train 0.127, test 0.095
degree  5: train 0.110, test 0.066
degree  9: train 0.090, test 0.103
degree 13: train 0.053, test 0.412
```

Takeaways

- Data = signal + noise; models should learn the signal.
- Underfitting misses the pattern; overfitting learns the noise.
- Training error always falls with complexity; only unseen data reveal overfitting.
- More data, regularisation, early stopping and physics all fight overfitting.

Check yourself

1. Overfitting means the model…
   (a) Is too simple · (b) Fits the training noise and fails on new data · (c) Has too little data to run · (d) Is physically correct
2. Which error reveals overfitting?
   (a) Training error · (b) Error on unseen data · (c) The number of parameters · (d) The learning rate
3. Regularisation works by…
   (a) Adding noise to the labels · (b) Penalising large weights in the loss · (c) Removing the test set · (d) Increasing the learning rate
4. How does physics reduce overfitting?
   (a) It adds data · (b) The equation residual penalises unphysical wiggles · (c) It removes noise from sensors · (d) It is not related

Answers: 1 (b) Learns the noise.; 2 (b) Test on unseen data.; 3 (b) Discourages extreme weights.; 4 (b) Physics is a regulariser.

Words

- **Noise**: Random variation in measurements that does not repeat.
- **Underfitting**: A model too simple to capture the pattern.
- **Overfitting**: A model so flexible that it fits the noise.
- **Validation set**: Data held back to choose models and settings.
- **Regularisation**: A penalty in the loss that discourages overly complex solutions.

Reading

- Hastie, Tibshirani and Friedman: The Elements of Statistical Learning (free): Chapter 7 on model assessment. https://hastie.su.domains/ElemStatLearn/
- Google: Machine Learning Crash Course, overfitting: Short interactive lessons. https://developers.google.com/machine-learning/crash-course/overfitting
- Karniadakis et al. (2021): Physics as a regulariser, in context. https://doi.org/10.1038/s42254-021-00314-5

1.7 Surrogates and emulators: fast stand-ins

35 minutes · Methods

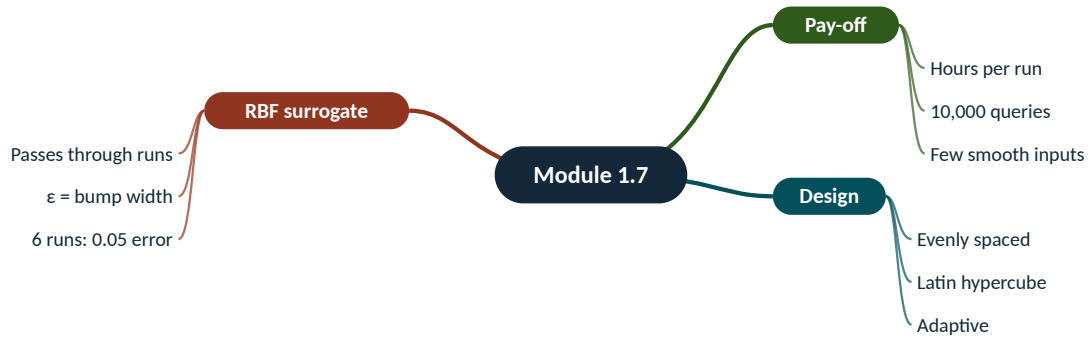
A surrogate learns the input–output map of a slow simulator from a few runs, then answers new questions almost instantly. Where you sample and what you assume about smoothness decide its accuracy.

By the end you can

- explain what a surrogate model is and when it pays off
- build a simple surrogate by interpolating a few runs
- describe how the number and placement of runs affect accuracy
- give transport examples of expensive models worth emulating

Overview

Many scientific models are slow. A detailed traffic simulation of a city, a climate and energy model such as GCAM, or a crash simulation can take hours for a single run. Yet we often want thousands of runs: to optimise a design, explore policies, or quantify uncertainty. A surrogate model, also called an emulator or metamodel, solves this. We run the expensive model at a few carefully chosen inputs, then fit a fast model to these input–output pairs. The surrogate answers new questions in milliseconds. Common surrogates include radial basis functions, Gaussian processes, polynomial chaos and neural networks. Accuracy depends on how many runs we make, where we place them, and how smooth the true relationship is. Placing runs evenly, or using space-filling designs such as Latin hypercubes, usually beats random placement. A good surrogate also says where it is unsure, so we know where to run the full model next. In this module you will emulate an expensive function with a handful of runs and see how placement and smoothness matter.



When surrogates pay off

If one run takes an hour and a study needs 10,000 runs, the full model needs more than a year of computing. A surrogate built from 20 runs answers the rest in seconds. Surrogates pay off when runs are expensive, inputs are few (roughly up to 20), and the response is reasonably smooth.

Interpolating with radial basis functions

A radial basis function (RBF) surrogate adds a bump centred at each run: $\hat{s}(x) = \Sigma\, w_j \exp(-(\varepsilon(x - x_j))^2)$. The weights are chosen so that $\hat{s}$ passes exactly through every run. ε sets how wide the bumps are: too narrow and the surrogate dips between runs; too wide and the system becomes ill-conditioned. In the explorer, 6 evenly spaced runs give an error of about 0.05, and 10 runs about 0.006.

TRY Try it

Use 6 runs and change ε from 0.3 to 6. Which values give a smooth, accurate surrogate? Then switch to random placement.

Designing the runs

Where you run matters as much as how many. Evenly spaced or **Latin hypercube** designs cover the input space; random designs leave gaps. **Adaptive** (active-learning) designs run the full model where the surrogate is most uncertain, which is where Gaussian processes (module 2.6) help. In Go 3 you will emulate a GCAM transport output this way.

GCAM GCAM link

Integrated assessment models such as GCAM are classic candidates for emulation: each scenario takes hours, and policy analysis needs thousands. See module 3.5.

LAB A radial basis function surrogate

More runs, smaller error, for a smooth function.

```
4 runs: surrogate error 0.5820
 6 runs: surrogate error 0.0522
10 runs: surrogate error 0.0060
```

Takeaways

- A surrogate is a fast model trained on a few runs of a slow one.
- It pays off when runs are expensive and many queries are needed.
- Accuracy depends on the number and placement of runs and on smoothness.
- Uncertainty-aware surrogates tell you where to run the full model next.

Check yourself

1. A surrogate model is…
   (a) A more detailed simulator · (b) A fast approximation learned from runs of a slow model · (c) A data cleaning step · (d) A type of sensor
2. Surrogates pay off most when…
   (a) Runs are cheap · (b) Runs are expensive and many queries are needed · (c) There is one query · (d) Inputs number in the thousands
3. Compared with random placement, evenly spaced runs usually…
   (a) Leave more gaps · (b) Cover the input space better · (c) Need more runs · (d) Are less accurate

4. Active learning chooses the next run where…
   (a) The surrogate is most certain · (b) The surrogate is most uncertain · (c) The input is zero · (d) Runs are cheapest

Answers: 1 (b) A fast stand-in.; 2 (b) Slow model, many questions.; 3 (b) No gaps.; 4 (b) Learn where unsure.

Words

- **Surrogate model**: A fast approximate model trained on input–output pairs of a slower model; also emulator or metamodel.
- **Radial basis function**: A function that depends only on distance from a centre, used as a building block for interpolation.
- **Design of experiments**: The choice of inputs at which to run a model or experiment.
- **Latin hypercube**: A space-filling design that spreads runs across each input's range.
- **Active learning**: Choosing new runs where the current model is most uncertain.

Reading

- Forrester, Sóbester and Keane (2008): Engineering Design via Surrogate Modelling: A practical textbook. https://doi.org/10.1002/9780470770801
- Gramacy (2020): Surrogates (free online book): Gaussian process surrogates with code. https://bobby.gramacy.com/surrogates/
- Kennedy and O'Hagan (2001): Bayesian calibration of computer models: The classic on emulators and calibration. https://doi.org/10.1111/1467-9868.00294

1.8 Physical AI: sense, estimate, decide, act

40 minutes · Physical AI

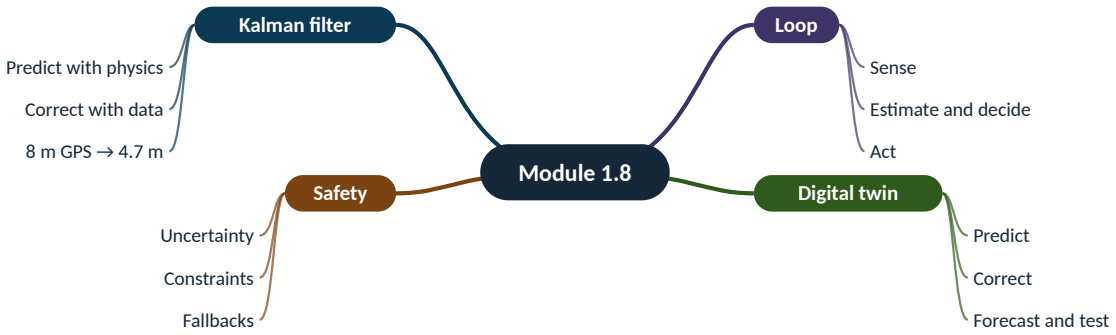
Physical AI closes a loop: sensors measure the world, filters estimate its state, models predict, controllers act, and the world responds. Physics-based estimation such as the Kalman filter is the foundation.

By the end you can

- describe the sense–estimate–decide–act loop of Physical AI
- explain how a Kalman filter blends a physics prediction with noisy measurements
- tune the filter's trust in model and sensor
- explain what a digital twin is and why safety matters for Physical AI

Overview

Physical AI is artificial intelligence with a body. An automated vehicle, a delivery robot or an adaptive traffic signal must repeat a loop many times a second: sense the world, estimate its true state, decide what to do, and act. Sensors are noisy. GPS jumps by several metres, radar misses targets, cameras are fooled by glare. So the first job is estimation. The Kalman filter, invented in 1960 and used in the Apollo missions, is the classic tool. It predicts where the vehicle should be using physics, position plus speed times time, and then corrects that prediction with the measurement, weighting each by how much it trusts it. If it trusts the model too much, it lags behind real changes. If it trusts the sensor too much, it just follows the noise. The same predict-and-correct idea powers digital twins, which keep a model of a road or a city in step with live data. Because Physical AI acts in the real world, safety, uncertainty and physics constraints are essential. In this module you will tune a Kalman filter tracking a car.



The loop

Every Physical AI system runs a loop: **sense → estimate → decide → act**. An adaptive cruise control does this about 10 to 100 times a second: radar measures the gap, a filter estimates the gap and relative speed, a controller chooses an acceleration, the engine and brakes apply it, and the gap changes.

The Kalman filter

The Kalman filter (Kalman 1960) alternates two steps. **Predict**: use physics, $x_{new} = x + v\,\Delta t$, and grow the uncertainty. **Update**: compare with the measurement and move the estimate towards it by the Kalman gain K, which is large when the sensor is trusted and small when the model is trusted. In the explorer, with GPS noise of 8 m, a well-tuned filter cuts the error to about 4.7 m; trusting the model too much (tiny q) makes it lag (7.7 m).

TRY Try it

Set q very small, then very large. Which setting lags behind the car, and which just follows the GPS noise?

Digital twins

A **digital twin** is a model kept continuously in step with a real system by data, using the same predict-and-correct idea at larger scale. A traffic digital twin of a corridor runs a traffic-flow model and corrects it with detector and probe data, so operators can see the current state, forecast the next minutes and test actions safely. You will build one in module 3.7.

Safety and trust

A wrong prediction in Physical AI can cause a crash. Systems therefore need uncertainty estimates (how sure is the estimate?), hard physical constraints (a car cannot decelerate faster than its tyres allow), fallbacks when sensors fail, and testing in simulation before the road. Standards such as ISO 26262 and ISO 21448 (SOTIF) cover functional safety of automated vehicles.

RESEARCH Research angle

Combining learned perception (neural networks on camera images) with physics-based filtering and control is a major research direction for automated vehicles and robots.

LAB A one-dimensional Kalman filter

Predict with physics, correct with the measurement, weighted by the Kalman gain.

```
GPS error 8.2 m, Kalman error 5.0 m
```

Takeaways

- Physical AI repeats a sense–estimate–decide–act loop.
- The Kalman filter blends a physics prediction with noisy measurements.
- Tuning sets the trust in model versus sensor: too much model lags, too much sensor follows noise.
- Digital twins and safety layers extend the same ideas to whole systems.

Check yourself

1. The Kalman filter's two steps are…
   (a) Train and test · (b) Predict with a model, then correct with a measurement · (c) Sense and act · (d) Encode and decode
2. If the filter trusts its model too much, the estimate…
   (a) Follows every noisy reading · (b) Lags behind real changes · (c) Becomes perfect · (d) Diverges to infinity always

3. A digital twin is…
   (a) A backup server · (b) A model kept in step with a real system by live data · (c) A second vehicle · (d) A video recording
4. Why is safety central to Physical AI?
   (a) It runs slowly · (b) Its actions have physical consequences · (c) It never uses data · (d) Regulations forbid AI

Answers: 1 (b) Predict, update.; 2 (b) It ignores the data.; 3 (b) Predict and correct at scale.; 4 (b) Mistakes cause harm.

Words

- **Physical AI**: AI that perceives and acts in the physical world.
- **State estimation**: Inferring the true state of a system from noisy measurements and a model.
- **Kalman filter**: An algorithm that optimally blends model predictions and measurements for linear systems with Gaussian noise.
- **Kalman gain**: The weight given to a new measurement relative to the prediction.
- **Digital twin**: A model continuously updated with data from the real system it represents.

Reading

- Kalman (1960): A new approach to linear filtering and prediction problems: The original paper. https://doi.org/10.1115/1.3662552
- Labbe: Kalman and Bayesian Filters in Python (free): Interactive notebooks for beginners. https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python
- NVIDIA: What is Physical AI?: An industry overview of the term. https://www.nvidia.com/en-us/glossary/generative-physical-ai/

Questions for spaced practice

From the question bank: answer from memory, then check. Revisit after 1, 3, 7 and 14 days.

1.1 What Physical AI and scientific machine learning are

1. **Recall:** Define scientific machine learning in one sentence.
   → Machine learning that combines data with scientific knowledge such as physical laws, to build models that are accurate, general and trustworthy.
2. **Explain:** Why does a pure data model fail when extrapolating?
   → It only learned patterns inside the range of its data and has no built-in reason to behave sensibly outside it.
3. **Explain:** What extra demands does Physical AI face compared with software AI?
   → It must respect physics, act in real time, and be safe when wrong, because its actions have physical consequences.
4. **Recall:** Name three families of SciML methods.
   → Physics-informed neural networks, equation discovery, operator learning and surrogates, hybrid models, and uncertainty quantification.
5. **Apply:** Give a transport example where data are sparse where it matters.
   → Crashes, extreme crowd densities, rare congestion events, or future climate and policy scenarios.
6. **Apply:** When might a pure physics model be enough?
   → When the laws are well known and parameters are measurable, such as projectile motion or simple heat flow, and details beyond the model do not matter.

1.2 Differential equations: the language of physics

1. **Recall:** What does a differential equation tell you?
   → How fast a quantity changes, as a function of its current state (and sometimes time).
2. **Calculate:** Decay with k = 0.5, y0 = 1, one Euler step of 0.5 s. What is y1?
   → $y1 = 1 + 0.5 \times (-0.5 \times 1) = 0.75$.
3. **Explain:** Why can Euler's method blow up?
   → It extrapolates along the current slope; with large steps or stiff/oscillatory problems the errors grow each step instead of shrinking.
4. **Recall:** What is the difference between an ODE and a PDE?
   → An ODE has one independent variable (usually time); a PDE has several, such as space and time.
5. **Apply:** Give a transport example of a PDE.
   → The LWR traffic-flow model, air pollution dispersion, or heat in pavements.
6. **Apply:** How do you check a numerical solution is accurate enough?
   → Halve the step size; if the answer barely changes, the solution has converged.

1.3 Neural networks from scratch

1. **Recall:** Write the computation of one tanh neuron.
   → $a = \tanh(w \cdot x + b)$: multiply inputs by weights, add the bias, apply tanh.
2. **Explain:** What is the universal approximation idea?
   → A network with enough hidden neurons can approximate any continuous function on a bounded range, by adding many soft steps.
3. **Calculate:** Parameters of a 1–20–1 network with tanh hidden layer?
   → $3 \times 20 + 1 = 61$.
4. **Explain:** Why do smooth networks struggle with sharp jumps?
   → Each neuron is a smooth soft step, so representing a sharp jump needs very steep or many neurons; otherwise the jump

gets blurred.

5. **Apply:** Name one way a network can break physics in transport.
   → Predicting negative speeds, flows above capacity, or vehicles appearing from nowhere.
6. **Recall:** What three things does a network need to learn?
   → Training data, a loss function and an optimiser.

1.4 Learning as optimisation: loss, gradients, autodiff

1. **Recall:** Write the gradient descent update.
   → $w \leftarrow w - \eta \nabla L(w)$, with η the learning rate.
2. **Calculate:** $L = (w - 3)^2$, $w = 0$, $\eta = 0.1$. Next w?
   → $w = 0 - 0.1 \times 2 \times (0 - 3) = 0.6$.
3. **Explain:** Why might gradient descent stop in a poor solution?
   → It can settle in a local minimum, a valley that is not the deepest, because every nearby step goes uphill.
4. **Explain:** What does momentum do?
   → It adds a fraction of the previous update to the current one, smoothing the path and helping to roll past small bumps.
5. **Explain:** Why is automatic differentiation essential for physics-informed learning?
   → It gives exact derivatives of the network output with respect to inputs such as time and space, which the physics residual needs.
6. **Apply:** Training loss is jumping up and down wildly. What do you try first?
   → Lower the learning rate (and check the data scaling).

1.5 Physics-informed learning: the core idea

1. **Recall:** Write the two parts of a physics-informed loss.
   → A data misfit term at measurements plus λ times the squared equation residual at collocation points.
2. **Explain:** Why are collocation points cheap?
   → They need no measurements: we only evaluate the model and its derivatives there, so we can place many anywhere.
3. **Explain:** Why does the residual need automatic differentiation?
   → The residual contains derivatives of the model output with respect to inputs, such as $d\hat{y}/dt$, which autodiff computes exactly.
4. **Apply:** Your PINN fits the data but the residual stays large. What might you change?
   → Increase the physics weight, add collocation points, or rescale the loss terms.
5. **Explain:** When is a classical solver better than a PINN?
   → For well-posed forward problems with known parameters and no data, where meshes and solvers are fast and accurate.
6. **Apply:** Give a transport inverse problem.
   → Estimating the free-flow speed or jam density of a road from sparse detector data and the LWR model.

1.6 Data, noise and overfitting

1. **Recall:** Define underfitting and overfitting.
   → Underfitting: model too simple, misses the pattern. Overfitting: model too flexible, fits noise and fails on new data.
2. **Explain:** Why does training error always fall as models get more flexible?
   → A more flexible model can always fit the training points at least as well, including their noise.
3. **Apply:** Your traffic model has 2% training error and 30% test error. Diagnose.
   → Overfitting: simplify, regularise, add data or add physical constraints.
4. **Explain:** What does regularisation add to the loss?
   → A penalty such as λ times the sum of squared weights, discouraging extreme values.
5. **Explain:** Why is physics a good regulariser?
   → Real systems obey laws that forbid many shapes noise would create; penalising the residual removes them.

6. **Apply:** How should you use a test set?
   → Keep it untouched until the end and use it once to report final performance.

1.7 Surrogates and emulators: fast stand-ins

1. **Recall:** What is a surrogate model?
   → A fast approximate model trained on a few input–output pairs from an expensive model.
2. **Explain:** When is building a surrogate worth it?
   → When each full run is expensive, many queries are needed, inputs are few and the response is fairly smooth.
3. **Explain:** What does the shape parameter ε do in an RBF surrogate?
   → It sets how wide each bump is: too narrow dips between runs, too wide makes the system ill-conditioned.
4. **Apply:** You can afford 10 GCAM runs over carbon prices 0–400 $/t. Where do you run them?
   → Spread evenly (or space-filling), then add runs adaptively where the surrogate is least certain.
5. **Recall:** Name three kinds of surrogate.
   → Radial basis functions, Gaussian processes, polynomial chaos, neural networks.
6. **Explain:** Why is surrogate uncertainty useful?
   → It shows where the surrogate may be wrong, guiding where to run the full model next and how much to trust answers.

1.8 Physical AI: sense, estimate, decide, act

1. **Recall:** Name the four steps of the Physical AI loop.
   → Sense, estimate, decide, act.
2. **Explain:** How does the Kalman filter combine model and measurement?
   → It predicts with the model, then moves the prediction towards the measurement by the Kalman gain, which depends on their relative uncertainties.
3. **Apply:** Your filter follows every GPS jump. What do you change?
   → Trust the model more: reduce the assumed process noise q or increase the assumed measurement noise R.
4. **Explain:** What is a digital twin?
   → A model of a real system that is continuously corrected with live data, used to monitor, forecast and test actions.
5. **Recall:** Name one safety standard for automated vehicles.
   → ISO 26262 (functional safety) or ISO 21448 (SOTIF).
6. **Apply:** Give a transport example of the sense–estimate–decide–act loop.
   → Adaptive cruise control, an adaptive traffic signal, or a lane-keeping system.